

Studio Model Compiler: Technical Manual

Documentation Project

March 31, 2026

Contents

1	System Overview and Main Execution	3
1.1	Introduction	3
1.2	The Execution Pipeline	3
1.3	Command Line Interface	3
2	Memory Management and Script Parsing	4
2.1	Memory Allocation Strategy	4
2.2	The Script Stack	4
2.2.1	Data Structure: <code>script_t</code>	4
2.2.2	Stack Mechanics	4
2.3	Tokenization State	5
3	Internal Data Representation	6
3.1	The Skeletal Hierarchy	6
3.2	Geometry and Meshes	6
3.2.1	Vertex and Normal Management	6
3.3	The Script Command Processor	6
3.3.1	Primary Directives	6
3.4	Coordinate Transformations	7
4	The Skeletal Animation System	8
4.1	Bone Hierarchy and Transformation	8
4.1.1	Bone Definition	8
4.2	Animation Sequences (<code>\$sequence</code>)	8
4.2.1	Sequence Groups	8
4.3	Unified <code>.mdl</code> File System	8
4.3.1	Blending and Interpolation	9
4.4	The Math of Motion	9
4.4.1	Matrix Concatenation	9
5	Textures and Skinning	10
5.1	Texture Management	10
5.1.1	The <code>lookup_texture</code> Function	10
5.2	Skin Families (<code>\$texturegroup</code>)	10
5.2.1	Implementation	10
5.3	Mesh-Texture Mapping	10
6	The Scripting Interface (QC Commands)	11
6.1	The Command Dispatcher	11
6.1.1	Lexical Analysis	11
6.2	Geometry Directives	11
6.3	Environmental and Global Settings	11

6.4	Collision and Optimization	12
7	Lexical Analysis and Tokenization	13
7.1	The Tokenizing Engine	13
7.1.1	The <code>GetToken</code> Function	13
7.2	Parsing Logic: The "UnGet" Mechanism	13
7.3	Numeric Conversion	13
8	Texture Alignment and Normalization	14
8.1	Hardware Compatibility and Resizing	14
8.2	Coordinate Transformation (The Quake Legacy)	14
8.3	Texture Deduplication and Replacement	14
8.3.1	Special Case: Chrome Materials	14
9	Rotations and Animation Compression	15
9.1	Trigonometric Rotation Matrices	15
9.1.1	Rotation Order	15
9.2	Animation Data Compression	15
9.2.1	The Compression Algorithm	15
9.3	Automated Sequence Grouping	16
10	Binary Serialization and File Layout	17
10.1	The Studio Header (<code>studiohdr_t</code>)	17
10.2	Memory Alignment and Padding	17
10.3	Sequence Splitting and External Files	17
10.4	Final Compilation Statistics	18
A	Command Line Reference	19
A.1	Flag Summary	19
B	Error Handling and Diagnostics	20
B.1	The <code>Error()</code> Function	20
B.2	Common Error Messages	20
B.3	Build Statistics	20

Chapter 1

System Overview and Main Execution

1.1 Introduction

The Studio Model Compiler is a command-line utility designed to process human-readable script files (`.qc`) and compile them into optimized binary model formats. It handles geometry transformation, texture mapping, and sequence group management.

1.2 The Execution Pipeline

The program follows a strictly linear compilation pipeline. When the utility is executed, it moves through several distinct phases:

- **Initialization:** Sets default environmental constants such as gamma (1.8) and default rotation (90°).
- **Argument Parsing:** Processes command-line flags to override default behaviors (e.g., flipping triangles or setting texture replacements).
- **Script Loading:** The `LoadScriptFile` function brings the ASCII instructions into memory.
- **Compilation:** This is the "heavy lifting" phase where the model is cleared, parsed, and simplified.
- **Serialization:** The final `WriteFile` call outputs the binary data to the disk.

1.3 Command Line Interface

The utility uses a standard flag-based interface. Key parameters include:

- `-t`: Texture replacement.
- `-a`: Normal blend angle (converted internally to a cosine value for calculation).
- `-i`: Ignore non-critical warnings during compilation.

Chapter 2

Memory Management and Script Parsing

2.1 Memory Allocation Strategy

The compiler utilizes a "load-to-memory" approach for file I/O. Rather than reading scripts line-by-line from the disk, the `LoadFile` function determines the total file size, allocates a single contiguous buffer via `malloc`, and reads the entire content into RAM.

- **Null Termination:** The buffer is allocated with an extra byte (`length + 1`) to ensure the script is null-terminated, allowing standard C string functions to operate safely on the data.
- **Safety Wrappers:** The use of `SafeOpenRead` and `SafeRead` indicates a robust error-handling layer that likely terminates the program if a file is missing or unreadable.

2.2 The Script Stack

To support modularity, the compiler implements a recursive script loading system using a stack of `script_t` structures. This allows the primary `.qc` file to include sub-scripts.

2.2.1 Data Structure: `script_t`

The `script_t` structure tracks the state of an active script:

- **buffer:** The start of the file in memory.
- **script_p:** A pointer to the current parsing position (the "cursor").
- **end_p:** A pointer to the end of the buffer to prevent memory overflows.
- **line:** An integer tracking the current line number for error reporting.

2.2.2 Stack Mechanics

The system supports a maximum depth of 8 nested files (`MAX_INCLUDES`).

1. **Initialization:** `LoadScriptFile` sets the global `script` pointer to the base of the stack.
2. **Pushing:** When `AddScriptToStack` is called, the stack pointer increments, and the new file is loaded into the next available slot.

3. **State Management:** Each level of the stack maintains its own line counter and filename, ensuring that when a sub-script finishes, the compiler can return to the exact previous position in the parent script.

2.3 Tokenization State

The parser maintains a global state for the lexical analyzer:

- **token:** A buffer containing the current word or symbol being processed.
- **tokenready:** A boolean flag used for "look-ahead" parsing (`UnGetToken`), allowing the parser to put a token back if it needs to re-evaluate a logic branch.

Chapter 3

Internal Data Representation

3.1 The Skeletal Hierarchy

The compiler views a model primarily as a hierarchy of bones. This allows for skeletal animation where moving a parent bone (like an arm) automatically transforms its children (the fingers).

- **s_bonetable_t:** This is the master definition for a bone. It stores the bone's name, its parent index, and its "hitbox" boundaries for collision detection.
- **s_node_t:** A simplified structure used during parsing to build the tree-like relationship between different parts of the model.
- **Bone Controllers:** The `s_bonecontroller_t` structure allows for dynamic, program-controlled bone movements, such as a character's head following a target or a mouth opening for speech.

3.2 Geometry and Meshes

The physical shape of the model is broken down into a hierarchy: `Model` $\rightarrow$ `Mesh` $\rightarrow$ `Triangle`.

3.2.1 Vertex and Normal Management

To optimize memory and performance, the compiler uses a "Lookup" system. Instead of storing duplicate data, it uses the following functions:

- **lookup_vertex:** Checks if a vertex with the same coordinates and bone assignment already exists before creating a new one.
- **lookup_normal:** Uses a "Normal Blending" threshold (`normal_blend`) to determine if two surface normals are close enough to be merged, smoothing the appearance of the model.

3.3 The Script Command Processor

The `ParseScript` function acts as the central dispatcher for the compiler. It scans the `.qc` file for commands prefixed with a `$`, which control every aspect of the build.

3.3.1 Primary Directives

The manual categorizes these commands into functional groups:

Command	Function
\$modelName	Defines the output filename of the compiled model.
\$body	Loads the reference geometry (usually an .smd file).
\$sequence	Defines an animation, its FPS, and its looping behavior.
\$eyeposition	Sets the origin point for the character's "view" in-game.
\$texturegroup	Handles skin variations (e.g., different uniforms for one model).

Table 3.1: Core Scripting Directives

3.4 Coordinate Transformations

As noted in the initialization, the compiler frequently transforms data from "Source Space" to "Object Space."

- **Concatenation:** The `R_ConcatTransforms` function is used to multiply 3×4 matrices, effectively combining parent and child bone rotations into a single world-space transformation.
- **Scaling and Adjusting:** The `scale_vertex` and `adjust_vertex` functions apply the user-defined `$scale` and `$origin` offsets to every vertex during the "Grabbing" phase.

Chapter 4

The Skeletal Animation System

4.1 Bone Hierarchy and Transformation

The core of the Studio Model format is its skeletal structure. Every vertex in the model is "attached" to a bone, and the movement of that bone dictates the movement of its associated vertices.

4.1.1 Bone Definition

The structure `s_bonetable_t` defines the physical properties of each node in the skeleton:

- **Parenting:** Each bone stores a `parent` index. A value of `-1` indicates a root bone. This creates a directed acyclic graph (DAG) of transformations.
- **Hitboxes:** Bones often carry `bbmin` and `bbmax` vectors. These define the "hitbox" used by the game engine for collision detection and projectile impact.
- **Flags:** Bones can be flagged with `STUDIO_BONE_HAS_NORMALS` or other properties to optimize how the engine processes them at runtime.

4.2 Animation Sequences (\$sequence)

A "Sequence" is a collection of animation frames. The compiler processes these from SMD (Studio Model Data) files.

4.2.1 Sequence Groups

To manage memory, the compiler supports `s_sequencegroup_t`. This allows animations to be stored in separate binary files, which is critical for characters with hundreds of complex animations.

4.3 Unified .mdl File System

The compiler produces multiple binary files with the `.mdl` extension to manage memory limits and modularity. The distinction between these files is handled internally via magic numbers (headers) rather than file extensions.

- **Primary Model:** Contains the `studiohdr_t` with the IDST magic number. This is the main entry point for the engine.

- **IDSTUDIOSEQHEADER (IDSQ):** While these files still use the `.mdl` extension, they contain sequence data. This allows the engine to load animations on demand rather than keeping every movement for every character in memory at once.
- **Texture Containers:** Similarly, if textures are stripped into external files, they are saved as `.mdl` files, named the same plus a T at the end (e.g., `modelT.mdl`).

4.3.1 Blending and Interpolation

The compiler calculates how to transition between different states:

- **Linear Blending:** The `s_anim_t` structure stores encoded rotation and position data.
- **Bone Controllers:** These allow the engine to override a bone's rotation programmatically (e.g., a "look at" controller for a character's neck).

4.4 The Math of Motion

At the heart of the animation system is the `R.ConcatTransforms` function.

4.4.1 Matrix Concatenation

When the compiler "grabs" a frame of animation, it must calculate the global position of a bone by multiplying its local transformation matrix by its parent's world transformation matrix. Mathematically, for a child bone C and parent P :

$$M_{world_C} = M_{world_P} \times M_{local_C}$$

This ensures that if a shoulder rotates, the elbow and wrist follow correctly in 3D space.

Chapter 5

Textures and Skinning

5.1 Texture Management

The compiler manages textures through the `s_texture_t` structure. Unlike geometry, which is processed into a proprietary format, textures are typically referenced as external bitmaps or embedded blocks.

5.1.1 The `lookup_texture` Function

This function is responsible for ensuring that each texture is only loaded once.

- **Deduplication:** It scans the current `textureref` table for a matching filename.
- **Special Flags:** As noted in the source, certain naming conventions (like the "chrome" suffix) trigger internal flags such as `STUDIO_NF_CHROME`, which changes how the engine renders the material's reflectivity.

5.2 Skin Families (`$texturegroup`)

One of the powerful features of this compiler is the support for "Skin Families." This allows a single mesh to use multiple sets of textures without duplicating the geometry data.

5.2.1 Implementation

The `texturegroup[32][32][32]` array in `studiomdl.h` stores these mappings.

- **Base Skin:** The default set of textures (Family 0).
- **Sub-Groups:** Additional families that can be switched at runtime (e.g., changing a character's camouflage or team color).

5.3 Mesh-Texture Mapping

Each `s_mesh_t` structure contains a `skinref` index. This index does not point directly to a texture, but rather to an entry in the skin table, providing a layer of indirection that makes the skin-swapping system possible.

Chapter 6

The Scripting Interface (QC Commands)

6.1 The Command Dispatcher

The core of the compilation process is driven by the `ParseScript` function. It operates as a lexical dispatcher, reading tokens from the script buffer and routing them to specialized command functions. All script commands are prefixed with a `$` symbol.

6.1.1 Lexical Analysis

The parser uses a global `token` buffer. When `GetToken(false)` is called, the parser skips whitespace and comments, retrieving the next meaningful string. If the token starts with `$`, the dispatcher identifies it as a directive.

6.2 Geometry Directives

These commands define the physical structure and "Body Parts" of the model.

- `$modelname <name>.mdl`: Sets the output path. As previously noted, while internal headers may differ, all output files use the `.mdl` extension.
- `$body <name> <file>.smd`: Loads a reference mesh. The compiler immediately "grabs" the vertices and normals from the specified SMD file.
- `$bodygroup <name>`: Defines a collection of sub-models that can be toggled. For example, a character could have a "head" bodygroup with two options: "helmeted" and "bare."

6.3 Environmental and Global Settings

These commands adjust the global state of the model during the "Grab" phase:

- `$origin [x] [y] [z]`: Offsets the entire model from the world origin.
- `$scale [float]`: Multiplies all vertex coordinates by a scale factor. This is useful for correcting size discrepancies between the 3D authoring tool and the game engine.
- `$eyeposition [vec3]`: Defines the coordinates of the character's eyes. The engine uses this for line-of-sight calculations and camera placement in first-person views.

6.4 Collision and Optimization

- **\$bbox and \$cbox:** These commands manually define the Bounding Box and Clipping Box. If omitted, the compiler usually calculates these automatically based on the vertex extremes.
- **\$flags:** Sets bitwise integers in the model header to toggle engine-side features like "flat shading" or "no-shadows."

Chapter 7

Lexical Analysis and Tokenization

7.1 The Tokenizing Engine

The compiler does not process files as a raw stream of characters. Instead, it uses a Lexical Analyzer (Lexer) to break the input into "Tokens"—the smallest units of meaning (such as keywords, numbers, or strings).

7.1.1 The GetToken Function

This is the most frequently called utility in the compiler. It performs the following steps:

- **Whitespace Stripping:** Automatically skips spaces, tabs, and newlines.
- **Comment Handling:** Recognizes and ignores C-style comments (`//`), allowing developers to document their `.qc` and `.smd` files without interfering with the build.
- **String Grouping:** If it encounters a quotation mark (`"`), it captures everything until the closing quote as a single token, which is essential for handling file paths with spaces.

7.2 Parsing Logic: The "UnGet" Mechanism

A unique feature of this parser is the `tokenready` flag and the `UnGetToken()` function.

- **Look-Ahead Parsing:** Sometimes the compiler needs to read a word to decide what to do, but then "give it back" so a sub-function can read it again.
- **Implementation:** When `UnGetToken()` is called, it sets `tokenready = true`. The next call to `GetToken` will simply return the existing buffer instead of reading a new word from the file.

7.3 Numeric Conversion

Since 3D models rely heavily on coordinates, the Lexer is tightly integrated with `atof` (ASCII to Float).

- **Vertex Extraction:** When parsing an SMD, the lexer pulls three consecutive tokens and converts them into the `vec3_t` format (X, Y, Z).
- **Error Resilience:** If `GetToken` reaches the end of a file unexpectedly, it sets the `endofscript` flag, triggering a graceful exit rather than a crash.

Chapter 8

Texture Alignment and Normalization

8.1 Hardware Compatibility and Resizing

Modern (and legacy) graphics hardware often requires texture dimensions to follow specific alignment rules for efficient memory access. The `ResizeTexture` function ensures that the compiled model adheres to these constraints.

- **Dword Alignment:** The compiler forces the texture width to be a multiple of 4 using bitwise logic: $(width + 3) \& \sim 3$. This ensures that each scanline of the texture starts on a 32-bit boundary.
- **UV Mapping Adjustments:** The `skintop` and `skinleft` values are calculated to create a local coordinate system for the mesh, effectively cropping the texture to the used area.

8.2 Coordinate Transformation (The Quake Legacy)

The `ResetTextureCoordRanges` function performs a critical transformation on the "T" (or V) coordinate of the UV mapping.

- **Inversion:** In many modeling tools, the vertical texture coordinate (T) starts from the bottom. However, following the Quake engine heritage, this compiler inverts the T coordinate:

```
pmesh->triangle[i][j].t = (max_t - min_t) - (current_t - min_t);
```
- **Normalization:** Texture coordinates are shifted from absolute pixel values to local offsets relative to the `min_s` and `min_t` of the specific mesh.

8.3 Texture Deduplication and Replacement

The compiler includes logic to handle "Replacement Textures." If a texture is identified as a replacement (often used for chrome or skin variants), it inherits the dimensions and coordinate ranges of its "parent" texture.

8.3.1 Special Case: Chrome Materials

As noted in the source code, if a texture is flagged with `STUDIO_NF_CHROME`, the compiler applies a fixed 64×64 (0 to 63) coordinate range. This is because chrome effects in this engine are procedurally generated based on the player's view angle rather than fixed coordinates on a bitmap.

Chapter 9

Rotations and Animation Compression

9.1 Trigonometric Rotation Matrices

The compiler represents 3D orientation using 3×4 transformation matrices. The function `AngleMatrix` is responsible for converting human-readable Euler angles (in degrees) into a format the engine can use for vertex transformation.

9.1.1 Rotation Order

The compiler applies rotations in a specific order to avoid Gimbal Lock and maintain consistency with the game engine's coordinate system.

- **Radian Conversion:** Input degrees are converted to radians using the formula: $rad = deg \times \frac{2\pi}{360}$.
- **Matrix Composition:** The resulting matrix combines Sine and Cosine values for Roll, Yaw, and Pitch to define the basis vectors for the bone's local space.

9.2 Animation Data Compression

To save memory, the compiler does not store every bone's position and rotation for every frame as raw floats. Instead, it uses a custom run-length encoding (RLE) scheme via the `mstudioanimvalue_t` structure.

9.2.1 The Compression Algorithm

The compression logic identifies "static" vs. "dynamic" bone movements:

- **Value Sampling:** The compiler compares bone positions across consecutive frames.
- **Run-Length Encoding:** If a bone remains stationary or moves linearly, the compiler groups these frames together into a "run," storing only the start value and the delta.
- **Quantization:** Floating point animations are often converted into integers or short-signed values to further reduce the size of the `.mdl` file.

9.3 Automated Sequence Grouping

As the model grows in complexity, the compiler must decide how to partition the data.

- **Size Thresholds:** The compiler monitors the `accumulated` size of animation data.
- **Dynamic Splitting:** If a single sequence group exceeds the `maxseqgroupsize` (defaulting to 1MB in the `main` function), the compiler automatically increments `numseqgroups` and begins a new data block.
- **Memory Alignment:** The `kalloc` function is used to ensure that compressed animation chunks are aligned in memory, preventing bus errors when the engine decompresses them during playback.

Chapter 10

Binary Serialization and File Layout

10.1 The Studio Header (`studiohdr_t`)

The serialization process begins with the construction of the primary header. This structure acts as a "Table of Contents" for the entire model, containing offsets to every other data block in the file.

- **Magic Number:** The header starts with `IDSTUDIOHEADER` ('IDST'), identifying the file type to the engine.
- **Bounding Boxes:** The `min`, `max`, `bbmin`, and `bbmax` vectors are written here, defining the physical space the model occupies for visibility culling and collision.
- **Offsets vs. Pointers:** Because the file is stored on disk, the compiler converts memory addresses (pointers) into relative offsets. This allows the game engine to "fix up" the pointers once the file is loaded into RAM.

10.2 Memory Alignment and Padding

To ensure optimal performance on 32-bit hardware, the compiler enforces strict memory alignment for every data block.

- **The ALIGN Macro:** The compiler uses a bitwise alignment macro:
`#define ALIGN(a) a = (byte *)(((int)((byte *)a + 3) & ~3)`
- **Function:** This ensures that every new section (Bones, Textures, Animations) starts on a 4-byte boundary, preventing unaligned memory access which would otherwise slow down the game engine.

10.3 Sequence Splitting and External Files

As noted in previous chapters, if a model's animation data exceeds the `maxseqgroupsize`, the writer initiates a multi-file output strategy.

- **Internal Labels:** Secondary files are initialized with the `IDSTUDIOSEQHEADER` ('IDSQ') and are assigned a label (e.g., "default").
- **The WriteAnimations Phase:** This function iterates through all compressed bone movements and writes them into the appropriate sequence group, ensuring that the primary `.mdl` only contains references to these external blocks.

10.4 Final Compilation Statistics

Upon successful completion, the writer prints a summary of the binary layout to the console. This is crucial for developers to monitor the "Memory Budget" of their models:

- **Bones:** Total bytes used for the skeletal hierarchy.
- **Animations:** The size of the RLE-compressed movement data.
- **Textures:** The combined size of the dword-aligned bitmaps.

Appendix A

Command Line Reference

A.1 Flag Summary

The `studiomdl` utility is controlled via the following command-line switches, as parsed in the `main` function:

- `-t [texture] [replacement]`: Replaces a specific texture during the build process.
- `-r`: Sets the `tag_reversed` flag, used for inverted geometry.
- `-n`: Sets `tag_normals`, forcing the compiler to ignore "bad" normals in the source SMD.
- `-f`: Disables triangle flipping (default is enabled).
- `-a [angle]`: Sets the normal blend threshold. Input is in degrees; stored internally as a cosine value.
- `-h`: Dumps hitbox data to the console during the write phase.
- `-i`: Ignore warnings; allows the compiler to proceed even if non-critical data is missing.
- `-g [size]`: Sets the maximum sequence group size in Kilobytes (e.g., `-g 1024`).

Appendix B

Error Handling and Diagnostics

B.1 The Error() Function

The compiler utilizes a "Fail-Fast" philosophy. When a critical error is encountered (e.g., a missing file or a stack overflow), the `Error()` function is called. This function:

1. Terminates the current process.
2. Reports the filename and line number (via `scriptline`) where the failure occurred.
3. Prevents the creation of a corrupted `.mdl` file.

B.2 Common Error Messages

- **"script file exceeded MAX_INCLUDES"**: Occurs if `.qc` files are nested more than 8 levels deep.
- **"unknown command"**: The Lexer found a token starting with `$` that is not defined in the `ParseScript` dispatcher.
- **"SafeRead: short read"**: The file on disk is smaller than expected or corrupted.

B.3 Build Statistics

The final output of `write.c` provides a memory breakdown. Use these figures to ensure your model stays within engine limits:

- **Bones**: Aim for fewer than 128 for compatibility with older computers.
- **Textures**: Keep within power-of-two or dword-aligned dimensions to avoid `ResizeTexture` overhead.